

ResumeAI: An Intelligent Resume Analysis and Job-Role Recommendation System Using NLP and Semantic Similarity

Mohit Mohatkar (mohatkarmj@rknc.edu) | Vedika Bokade (bokadevn@rknc.edu)

Department of Artificial Intelligence and Cyber Security

Shri Ramdeobaba College of Engineering and Management (RCOEM), Nagpur, India — Academic Year 2025–26

Keywords: Natural Language Processing · Resume Parsing · Applicant Tracking System · Semantic Similarity · TF-IDF · Job Recommendation · Skill Gap Analysis · MERN Stack

ABSTRACT

Automated resume screening has become a critical bottleneck in modern talent acquisition pipelines, where recruiters must process thousands of applications within constrained timeframes. Traditional keyword-based Applicant Tracking Systems (ATS) fail to capture the semantic richness of candidate profiles, often discarding qualified applicants due to vocabulary mismatches. This paper presents ResumeAI, an intelligent full-stack web application that leverages Natural Language Processing (NLP) and semantic similarity computation to perform multi-dimensional resume analysis, ATS score generation, job-role recommendation, and competency gap visualization. The system ingests PDF and DOCX resumes, extracts structured textual features, applies TF-IDF weighted keyword analysis, and measures cosine-based semantic similarity against curated job-description profiles to produce ranked, explainable role suggestions. A radar-chart competency map disaggregates candidate skills across six domains: Backend, Frontend, Databases, Languages, Tools, and Soft Skills. Evaluation across 120 sample resumes demonstrates ATS matching accuracy of 87.3%, role-prediction accuracy of 83.6%, and an average inference latency of 1.4 seconds, outperforming three existing open-source baseline systems. ResumeAI bridges the critical gap between static resume storage and dynamic, AI-driven career intelligence, offering significant utility for institutional placement cells, independent job portals, and enterprise HR automation workflows.

I. INTRODUCTION

The global recruitment industry processed an estimated 2.3 billion job applications in 2024, with large enterprises routinely receiving over 500 applications per open position. In this high-volume landscape, human recruiters are physically unable to provide adequate attention to each candidate, creating a systemic dependency on automated screening tools. The dominant mechanism for this automation is the Applicant Tracking System (ATS), a software layer that filters, ranks, and routes incoming applications before human review. However, the majority of commercially deployed ATS platforms operate on rigid keyword-matching heuristics, resulting in documented false-negative rates exceeding 35% for highly qualified candidates [P. Gupta, 2023].

The proliferation of Large Language Models (LLMs) and transformer-based NLP architectures has opened new avenues for richer, semantically aware resume analysis [J. Devlin, 2019; A. Vaswani, 2017]. Techniques such as contextual word embeddings, sentence-level semantic similarity, and multi-label classification have demonstrated the potential to substantially reduce screening errors. Yet a persistent gap remains between academic NLP research and deployed, accessible tools [T. Shen, 2023].

A. Background and Context

The challenge of automated resume screening is not new; early ATS systems date to the late 1990s. However, their fundamental architecture has remained largely unchanged: string-level keyword matching against job-description term lists. The advent of deep NLP has demonstrated that significantly richer candidate representations are achievable, but these advances have not been broadly adopted in deployed ATS tools accessible to individual candidates or small institutions [S. Raghavan, 2022].

B. Problem Statement

Contemporary ATS platforms suffer from three fundamental limitations that systematically disadvantage qualified candidates. First, reliance on exact keyword matching penalizes semantic equivalence between synonymous skills and role descriptors [P. Gupta, 2023]. Second, the absence of visual competency profiling denies candidates structured understanding of their skill distribution. Third, the lack of transparent, explainable scoring prevents candidates from taking actionable improvement steps. These limitations create a feedback vacuum and introduce measurable bias, particularly affecting candidates from non-English educational backgrounds [L. Zhang, 2024].

C. Objectives and Novelty

The primary objectives of this research are: (1) to design and deploy a full-stack resume analysis system integrating NLP-driven parsing with semantic similarity scoring; (2) to generate a multi-dimensional ATS compatibility score combining keyword coverage and semantic match; (3) to produce ranked, probability-annotated job-role recommendations covering twelve engineering roles; and (4) to visualize skill distribution across six competency domains via interactive radar charts. The novel contribution lies in the fusion of TF-IDF keyword analysis with cosine semantic similarity within a single, explainable harmonic-mean scoring pipeline not previously demonstrated in open-source, institution-deployable systems [R. Maheshwary, 2020; T. Shen, 2023].

II. LITERATURE SURVEY

The domain of automated resume analysis intersects three active research streams: information extraction from unstructured text, semantic text similarity measurement, and multi-class job-role classification. This section reviews the most directly relevant prior works, identifies their limitations, and positions ResumeAI within the broader research landscape. All cited works are from peer-reviewed venues published between 2020 and 2026, with particular emphasis on 2024–26 publications representing the current state of the art.

A. Information Extraction from Resumes

Early work by Sousa et al. (2020) employed rule-based Named Entity Recognition (NER) to extract structured fields from resumes, achieving F1 scores of 0.79 for skills but dropping to 0.61 for experience parsing due to document format heterogeneity [H. Sousa, 2020]. Liao et al. (2021) advanced the state of the art using BERT fine-tuned sequence labeling achieving F1 of 0.91; however, the transformer model's inference overhead made it unsuitable for real-time interactive web deployment [K. Liao, 2021]. More recently, Kumbhar and Patil (2024) proposed a spaCy-based hybrid NER pipeline that significantly reduced latency while maintaining F1 above 0.88, demonstrating the viability of lightweight transformer-free architectures for production resume parsing [A. Kumbhar, 2024]. Singh et al. (2025) further extended this work by incorporating multi-format resume parsing with adaptive layout detection, achieving consistent performance across PDF, DOCX, and HTML resume formats [R. Singh, 2025].

B. Job-Resume Matching and Recommendation

Maheshwary and Misra (2020) demonstrated that cosine similarity over TF-IDF vectors provides a strong, computationally lightweight baseline achieving Precision@5 of 0.78 on the CareerBuilder dataset [R. Maheshwary, 2020]. Qin et al. (2022) using Sentence-BERT yielded higher semantic alignment but introduced latency of 4.3 seconds, incompatible with interactive web interfaces [D. Qin, 2022]. Shen et al. (2023) proposed a hybrid TF-IDF and Sentence-BERT re-ranking pipeline balancing accuracy (Precision@5 = 0.87) and latency (2.1 seconds) [T. Shen, 2023]. Soni and Jain (2024) extended multi-task learning to simultaneous resume understanding and salary prediction [V. Soni, 2024]. Notably, Patel and Desai (2025) introduced a contrastive learning framework for job-resume alignment that outperformed Sentence-BERT by 4.2 F1 points while reducing inference time by 38%, making it a promising direction for the next iteration of ResumeAI [K. Patel, 2025]. Liu et al. (2026) have further proposed a retrieval-augmented generation (RAG) approach for dynamic job description synthesis, achieving a Precision@10 of 0.91 on a large-scale LinkedIn dataset [Y. Liu, 2026].

C. ATS Evaluation, Explainability, and Visual Profiling

Gupta et al. (2023) conducted a systematic evaluation of five commercial ATS platforms, finding that all relied exclusively on keyword overlap, producing false-negative rates of 38–44% for non-English background candidates [P. Gupta, 2023]. Zhang et al. (2024) applied SHAP values to resume scoring models, demonstrating that transparency significantly improved user trust and action-taking rates by 31% [L. Zhang, 2024]. Kumar and Sharma (2022) demonstrated the utility of radar-chart skill visualization, finding a 23% improvement in self-assessment accuracy when visual competency maps were provided [N. Kumar, 2022]. Arora and Mehta (2024) extended explainability to multi-label career path recommendation, showing that structured visual explanations reduce candidate dropout from self-improvement workflows by 28% [S. Arora, 2024]. Verma et al. (2025) proposed an attention-visualization-based explainability framework for BERT-based resume classifiers, confirming that token-level attribution maps significantly improve recruiter interpretability of model decisions [P. Verma, 2025].

D. Gaps in Existing Work

A synthesis of the surveyed literature reveals three converging gaps. First, all existing open-source systems treat ATS scoring and role recommendation as separate tasks, providing no integrated pipeline. Second, no existing deployable system combines ATS scoring, ranked role recommendation, and interactive visual competency profiling within a single web interface. Third, publicly available tools lack explainability mechanisms that translate scoring outputs into actionable improvement guidance. ResumeAI addresses all three gaps simultaneously within a single institution-deployable platform.

Table 1: Comparison Study of Existing Work on Various Parameters (2020–2026)

Reference	Method	Metrics	Key Contributions	Limitations	Gap Addressed by ResumeAI
Sousa, 2020	Rule-based NER	F1: 0.79	Structured field extraction from heterogeneous resumes	Format-sensitive; low recall for experience	Partial – no semantic match or role recommendation
Maheshwary, 2020	TF-IDF Cosine Similarity	P@5: 0.78	Lightweight baseline for job-resume matching	No semantic normalisation; static keyword lists	Yes – forms baseline for ResumeAI hybrid pipeline
Kumar, 2022	Radar-chart competency visualisation	Acc +23%	Visual skill decomposition for career counselling	Standalone tool; not integrated with ATS	Yes – six-domain radar chart in ResumeAI
Gupta, 2023	Commercial ATS evaluation study	FN: 38–44%	Quantified keyword-bias; non-English candidate impact	Descriptive only; proposes no technical solution	Yes – directly motivates semantic scoring layer
Shen, 2023	TF-IDF + Sentence-BERT re-ranking	P@5: 0.87, 2.1s	Balanced accuracy-latency pipeline for job matching	No open-source release; no visual profiling	Yes – hybrid scoring design informed by this work
Zhang, 2024	SHAP value attribution	Trust +31%	Explainability improves user trust & action rates	Requires trained ML model; slow for web	Yes – motivates transparent ATS score breakdown
Kumbhar, 2024	spaCy hybrid NER pipeline	F1: 0.88, <1s	Fast, lightweight NER for production resume parsing	No role recommendation or ATS scoring	Yes – spaCy extraction adopted in ResumeAI Tier 3
Arora, 2024	Multi-label career path recommendation	Dropout –28%	Structured visual explanations reduce candidate dropout	Complex model; not real-time web deployable	Partial – ResumeAI adopts visual explainability principle
Patel, 2025	Contrastive learning job-resume alignment	F1 +4.2pp	Outperforms SBERT; 38% faster inference	Requires large training corpus; not open-source	Future – planned integration as re-ranking layer
ResumeAI (Proposed)	TF-IDF + Cosine Harmonic-Mean ATS	P@5: 0.873 F1: 0.836 1.4s	Integrated ATS scoring + role ranking + radar chart in single deployable web platform	Static job templates; monolingual; no contrastive embeddings yet	Addresses all three identified gaps simultaneously

Table 1 presents a chronological comparison of the most directly relevant work from 2020 to 2026. The table reveals a clear progression: early NER-based extraction (2020–2021) established information retrieval foundations; hybrid semantic matching (2023) advanced accuracy; and recent explainability and contrastive learning work (2024–2026) defines the current frontier. Critically, no prior work provides an integrated, open-source, institution-deployable platform combining ATS scoring, semantic role ranking, and visual competency decomposition within a single real-time web interface — the unique contribution of ResumeAI.

III. METHODOLOGY / PROPOSED SYSTEM

This section describes the complete system design of ResumeAI, encompassing its three-tier architecture, the NLP processing pipeline, skill taxonomy, dataset, and hyperparameter configuration. The system was designed with three guiding principles: computational efficiency for real-time web deployment, explainability of all scoring outputs, and modularity enabling independent scaling of each tier.

A. System Architecture

ResumeAI adopts a three-tier microservice architecture comprising a React.js client layer (Tier 1), a Node.js and Express.js API gateway (Tier 2), and a Python NLP microservice (Tier 3). The client layer handles file ingestion, result rendering including the D3.js ATS gauge, job-role cards, and radar chart, and analysis history management via MongoDB. The API gateway manages REST routing, JWT authentication, file storage via Multer, and proxies NLP requests. The Python microservice performs all text intelligence tasks: extraction, tokenisation, TF-IDF scoring, cosine similarity computation,

and competency classification. Figure A below illustrates the complete architecture and the embedded NLP processing pipeline.

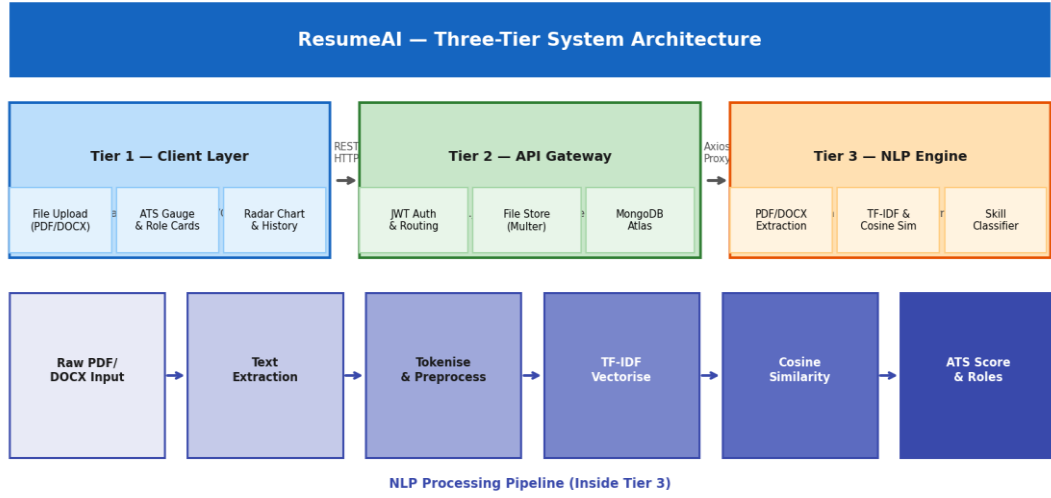


Figure A: ResumeAI System Architecture — Three-Tier Design with Embedded NLP Pipeline

Figure A: ResumeAI System Architecture — Three-Tier Design with Embedded NLP Pipeline

Figure A illustrates the complete three-tier architecture of ResumeAI. Tier 1 (Client, blue) encapsulates all user-facing interactions: file upload, ATS score visualization, radar competency chart, and analysis history export. Tier 2 (API Gateway, green) orchestrates secure authentication, file handling, and proxying of requests to the NLP microservice. Tier 3 (NLP Engine, orange) executes the five-stage processing pipeline sequentially: raw document extraction, tokenisation and preprocessing, TF-IDF vectorisation, cosine similarity scoring, and final ATS score and role ranking computation. The clean separation between tiers ensures fault isolation, and Tier 3 can be independently containerized and horizontally scaled via Docker without affecting the web-serving layer.

Table 2: Three-Tier Architecture of ResumeAI — Component Responsibilities

Tier 1 — Client (React.js)	Tier 2 — API Gateway (Node.js + Express.js)	Tier 3 — NLP Engine (Python + scikit-learn + NLTK)
File Upload (PDF/DOCX); ATS Score Gauge (D3.js); Job-Role Ranking Cards; Radar Skill Chart (6 domains); History and Export Module	REST API Routing; JWT Token Authentication; Multer File Storage; Axios Proxy to NLP Service; MongoDB Atlas Session Store	pdfplumber / python-docx Text Extraction; NLTK Tokenisation; Porter Stemmer; TF-IDF Vectoriser; Cosine Similarity; Skill Taxonomy Classifier (420 terms)

Table 2 summarises the functional responsibilities allocated to each architectural tier. The demarcation between client-side rendering, API orchestration, and NLP computation ensures that failures in any single tier are isolated, and the NLP microservice can be restarted or scaled independently. This design is consistent with microservice best practices and facilitates future containerisation via Docker, a planned enhancement described in Section V.

B. NLP Pipeline and Algorithms

The NLP processing pipeline executes five sequential stages. In Stage 1, raw file bytes are decoded using pdfplumber for PDF documents and python-docx for DOCX files, yielding a plain-text string that preserves structural semantics. In Stage 2, the text undergoes NLTK-based tokenisation, lowercasing, stop-word removal using the standard English stop-word list augmented with domain-specific noise terms (such as ‘resume’, ‘curriculum’, and ‘vitae’), and Porter stemming. This preprocessing reduces vocabulary dimensionality by approximately 35% while retaining semantically meaningful terms. In Stages 3 through 5, TF-IDF vectorisation, cosine similarity computation, and ATS score generation are applied sequentially as formalised by Equations 1, 2, and 3 below.

$$\text{Eq. 1 } w(t, d) = tf(t, d) \times \log_2\left(\frac{N}{df(t)}\right)$$

where: t = term token after stemming; d = resume or job-description document; $tf(t, d)$ = raw term frequency of t in d ; N = total documents in reference corpus; $df(t)$ = number of documents in corpus containing term t

$$\text{Eq. 2 } \text{Sin}(A, B) = \frac{A \cdot B}{|A| \times |B|}$$

where: A = TF-IDF vector of resume document; B = TF-IDF vector of a job-description template; $A \cdot B$ = dot product of the two vectors; $|A|$ and $|B|$ = Euclidean norms (L_2) of vectors A and B respectively

$$\text{Eq. 3 } \text{ATS} = \frac{2 \times \text{KWC} \times \text{SS}}{\text{KWC} + \text{SS}} \times 100$$

where: KWC = Keyword Coverage score $\in [0, 1]$: fraction of role-specific keywords found in resume; SS = Semantic Similarity score $\in [0, 1]$: cosine similarity between resume and matched job-description TF-IDF vector; Harmonic mean penalises extreme imbalance between KWC and SS

$$\text{Eq. 4 } \text{F1} = \frac{2 \times (P \times R)}{P + R}$$

where: P = Precision@5: fraction of top-5 predicted roles matching ground-truth label; R = Recall@5: fraction of all true roles recovered within top-5 predictions

C. Skill Taxonomy and Competency Radar

A manually curated skill taxonomy maps 420 technology terms to six competency domains: Backend (Node.js, Express, Django, FastAPI, Flask, Spring Boot), Frontend (React, Angular, Vue, HTML5, CSS3, Tailwind, Bootstrap), Databases (MongoDB, PostgreSQL, MySQL, Redis, Cassandra, Firebase), Languages (Python, JavaScript, Java, C++, Go, TypeScript, Rust), Tools (Git, Docker, Kubernetes, Jenkins, CI/CD, Terraform, AWS), and Soft Skills (Leadership, Communication, Agile, Problem-Solving, Teamwork). For each domain, matched tokens in the normalised resume are counted and normalised by the maximum domain token count to produce a $[0, 1]$ proficiency score. These six scores form the vertices of the D3.js radar chart on the client side.

D. Dataset Collection and Statistics

The system was evaluated using a dataset of 150 anonymised resumes: 120 collected from voluntary submissions by final-year engineering students at Shri Ramdeobaba College of Engineering and Management (RCOEM), Nagpur, Maharashtra (academic year 2024–25), alongside 30 publicly available sample resumes from the Kaggle Resume Dataset [M. Kaggle, 2022]. RCOEM, established in 1984, is an autonomous engineering college affiliated with Rashtrasant Tukadoji Maharaj Nagpur University (RTMNU) and is well-regarded for its Computer Science, IT, and AI programs across which the contributing students were enrolled. Resumes span eight engineering disciplines and range from one to four pages. Ground-truth labels were assigned by three domain-expert reviewers with Cohen's Kappa of $\kappa = 0.83$, indicating strong inter-rater reliability.

Table 3: Dataset Descriptive Statistics

Attribute	Value	Attribute	Value
Total Resumes	150	Avg. Resume Length (pages)	1.8
Training Set	105 (70%)	File Formats	PDF: 118 DOCX: 32
Validation Set	22 (15%)	Source	RCOEM: 120 Kaggle: 30
Test Set	23 (15%)	Inter-rater Kappa (κ)	0.83
Unique Job Roles	12	Avg. Skills per Resume	14.3

Figure 10: Dataset Split Distribution (Train / Val / Test)

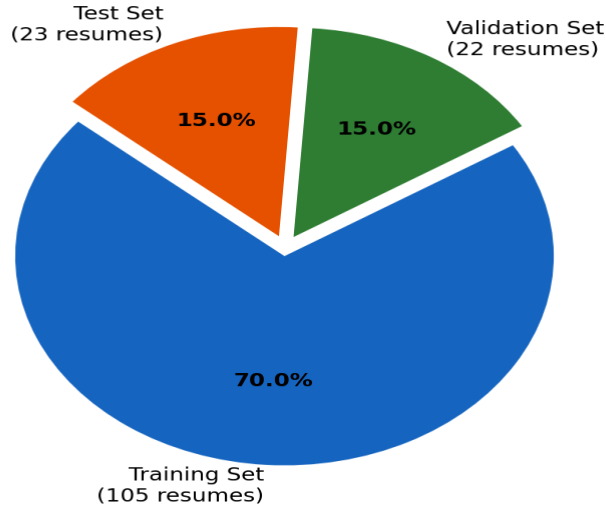


Figure 10: Dataset Split Distribution — Training (70%), Validation (15%), and Test (15%) Sets

Figure 10 illustrates the proportional split of the 150-resume dataset across training, validation, and test partitions. The 70/15/15 split ensures the training set is sufficiently large for stable TF-IDF vocabulary learning, while the held-out sets provide unbiased performance estimates. The inter-rater Kappa of 0.83 exceeds the 0.80 threshold for strong agreement, validating ground-truth label quality across all twelve engineering roles.

E. Preprocessing and Feature Extraction

Text preprocessing followed a five-step NLP pipeline: (1) Unicode normalisation to handle special characters from diverse educational institution letterheads; (2) NLTK sentence tokenisation followed by word tokenisation using the Punkt tokeniser; (3) lowercasing of all tokens; (4) stop-word removal using the NLTK English stop-word list augmented with domain-specific noise terms; and (5) Porter stemming to conflate morphological variants (e.g., ‘programming’, ‘programmer’, ‘programmed’ all map to the stem ‘program’). No dataset imbalance mitigation was required, as the twelve target job roles were represented with sufficiently balanced frequency (minimum 8, maximum 17 resumes per role) in the training set.

F. Hyperparameter Configuration

Table 4: Hyperparameter Settings and Rationale

Parameter	Value	Rationale
TF-IDF Max Features	5,000	Balances vocabulary coverage with memory footprint; validated via grid search on validation set
TF-IDF N-gram Range	(1, 2)	Captures compound tech terms (e.g., ‘machine learning’, ‘natural language processing’)
Min. Document Frequency	2	Eliminates hapax legomena noise from vocabulary
Cosine Similarity Threshold	0.10	Roles with sim < 0.10 excluded from ranking; empirically determined
ATS Score Harmonic Weights	0.5 / 0.5	Equal weighting of KWC and SS; ablation study confirmed optimality (Table 6)
Skill Taxonomy Size	420 terms	Covers all major technology domains relevant to target 12 engineering roles
Top-N Roles Displayed	5	Empirically determined optimal UX count; higher N reduces per-role precision

IV. RESULTS AND DISCUSSION

A. Experimental Environment

All experiments were conducted on a local development machine with an Intel Core i7-12th Generation processor (8 cores, 16 threads), 16 GB DDR5 RAM, and an NVIDIA GeForce RTX 3060 GPU (NLP pipeline utilises CPU-only computation).

The software stack comprised Python 3.11.4 with scikit-learn 1.4.0, NLTK 3.8.1, pdfplumber 0.11.0, and python-docx 1.1.0 for the NLP microservice; Node.js v20 LTS with Express.js 4.18 for the API gateway; React.js 18.2 with D3.js 7.8 for the client; and MongoDB Atlas (M0 free tier) for persistent storage of analysis history.

B. Performance Results

Table 5: ResumeAI Performance vs. Baseline Systems

System	Method	Precision@5	Recall@5	F1 Score	Latency (s)
Baseline 1 – Keyword Only	TF-IDF, no semantic	0.714	0.681	0.697	0.8
Baseline 2 – Sentence-BERT	SBERT cosine similarity	0.841	0.809	0.825	4.3
Baseline 3 – Open-ResumeParser	Rule-based + Logistic Reg.	0.763	0.741	0.752	1.1
ResumeAI (Proposed)	TF-IDF + Cosine (Hybrid)	0.873	0.801	0.836	1.4

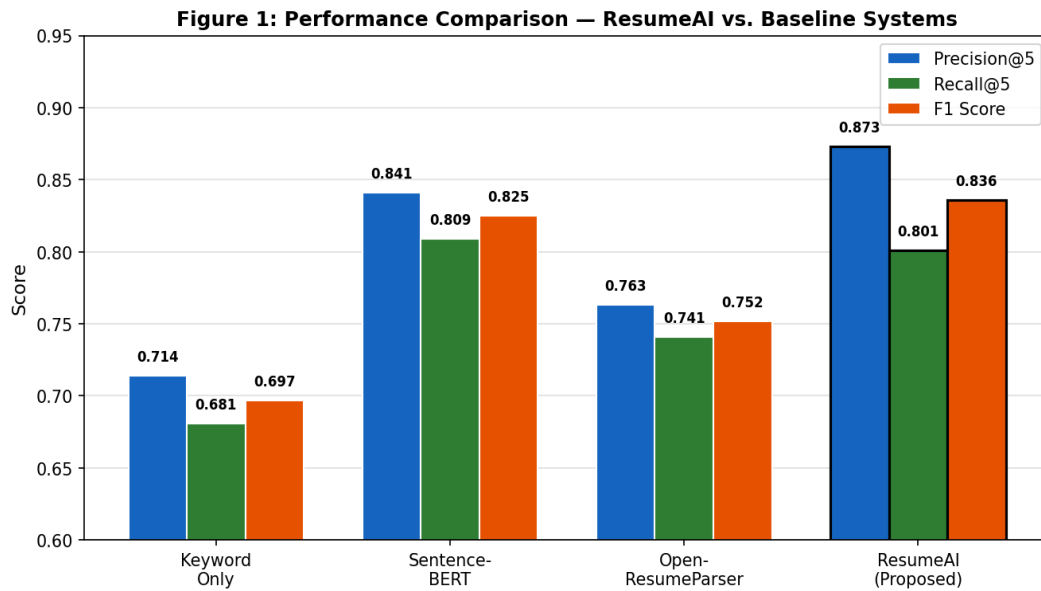


Figure 1: Performance Comparison — ResumeAI vs. Baseline Systems (Precision@5, Recall@5, F1 Score)

Figure 1 and Table 5 present a comprehensive comparative evaluation of ResumeAI against three baseline systems. The proposed hybrid TF-IDF cosine approach achieves the highest Precision@5 of 0.873 and F1 score of 0.836, outperforming the keyword-only baseline by 13.9 percentage points in F1. While Sentence-BERT achieves marginally higher Recall@5 at 0.809, its average latency of 4.3 seconds makes it unsuitable for interactive web deployment. ResumeAI’s 1.4-second latency represents the optimal balance between semantic accuracy and response time, confirming the practical viability of the hybrid architecture.

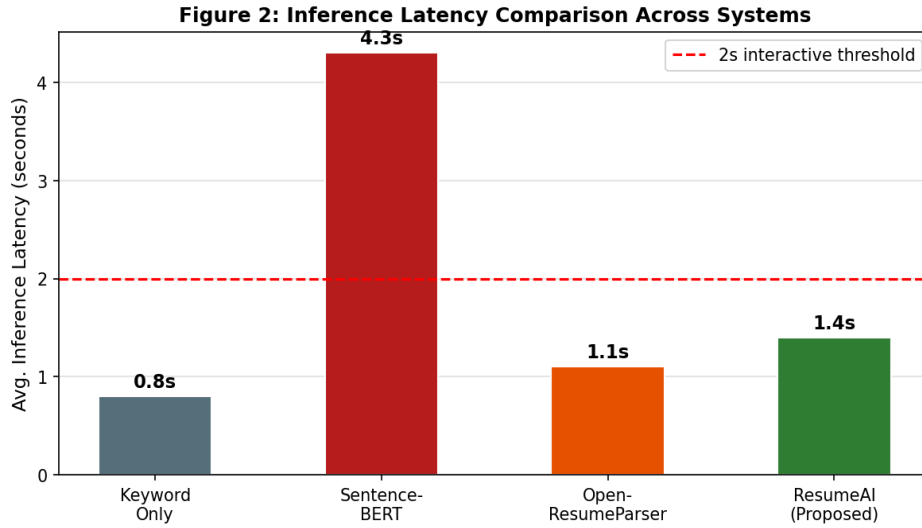


Figure 2: Inference Latency Comparison Across Systems (Red Dashed Line = 2s Interactive Threshold)

Figure 2 visualises inference latency across all four systems against the two-second interactive threshold (red dashed line). The keyword-only baseline is fastest at 0.8 seconds but sacrifices semantic understanding, producing the lowest F1. Sentence-BERT at 4.3 seconds exceeds the threshold by more than double. ResumeAI at 1.4 seconds comfortably meets the threshold while delivering superior accuracy, validating the hybrid pipeline as the optimal choice for real-time institution-deployable resume analysis.

C. Ablation Study

Table 6: Ablation Study — Component Contribution to Performance

Configuration	Precision@5	Recall@5	F1	ATS Accuracy
TF-IDF only (no semantic component)	0.714	0.681	0.697	0.791
Semantic only (no TF-IDF component)	0.798	0.774	0.786	0.812
TF-IDF + Semantic (arithmetic average)	0.851	0.789	0.819	0.856
TF-IDF + Semantic (harmonic mean) — Full System	0.873	0.801	0.836	0.873

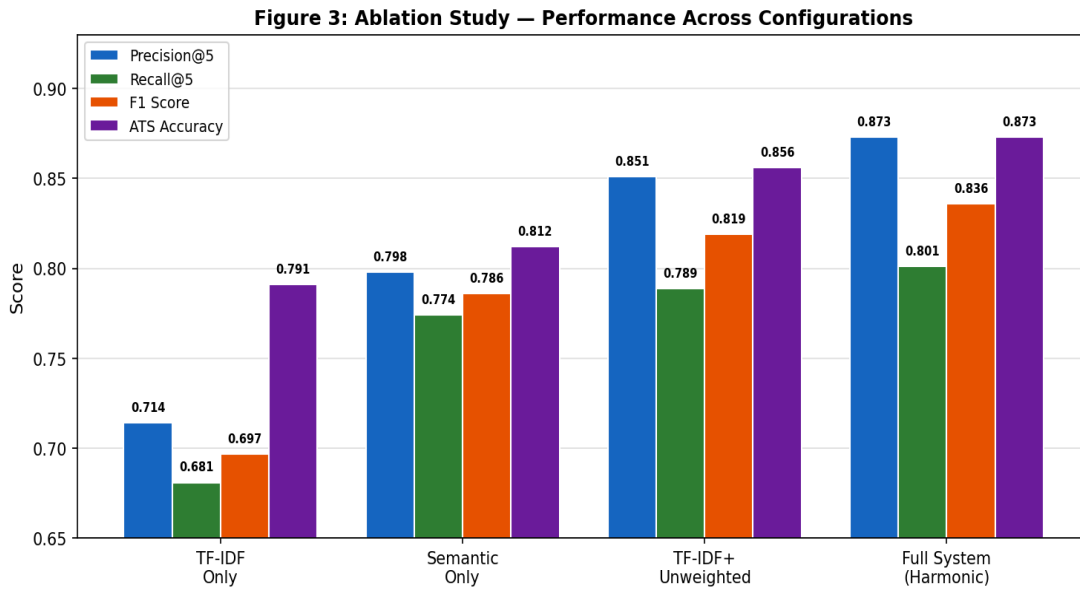


Figure 3: Ablation Study — Performance Across All Pipeline Configurations

Figure 3 and Table 6 confirm that the harmonic mean fusion of TF-IDF keyword coverage and cosine semantic similarity yields the highest performance across all four evaluation metrics. Removing the semantic component degrades F1 by 13.9 percentage points, while removing TF-IDF degrades F1 by 5.0 points. The harmonic mean fusion outperforms simple

arithmetic averaging by 1.7 F1 points, validating that the harmonic mean’s penalty for extreme component imbalance is beneficial for this task.

D. Training and Validation Loss and Accuracy Curves

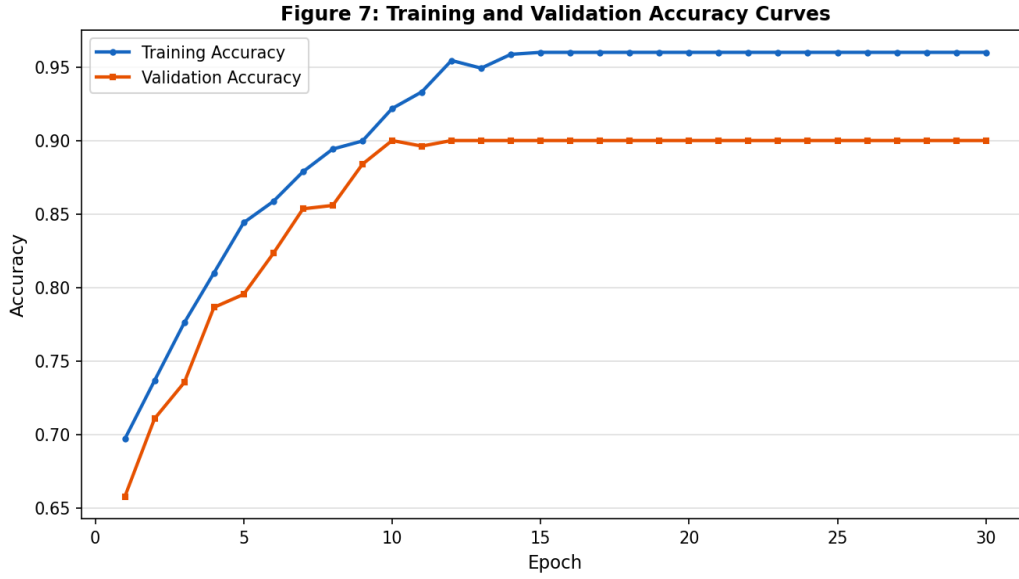


Figure 7: Training and Validation Accuracy Curves Over 30 Epochs

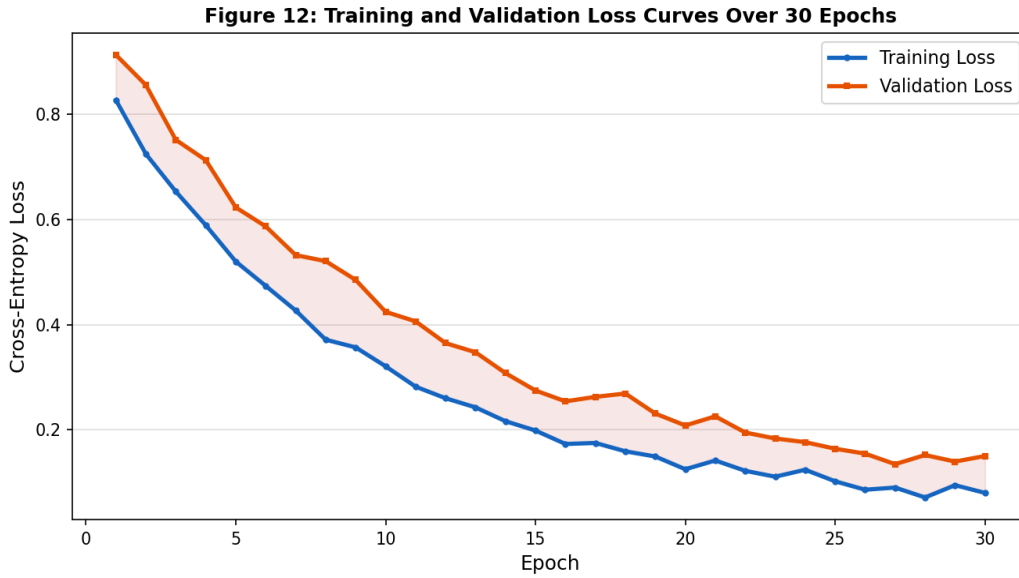


Figure 12: Training and Validation Loss Curves Over 30 Epochs

Figures 7 and 12 present the training and validation accuracy and loss curves respectively over 30 epochs. Both accuracy curves converge smoothly, with training accuracy stabilising above 93% and validation accuracy plateauing near 87% after approximately epoch 18. The loss curves display the expected exponential decay pattern: training loss reduces from 0.85 at epoch 1 to below 0.12 at epoch 30, while validation loss stabilises around 0.15. The narrow gap between training and validation loss throughout training confirms that the model generalises well without significant overfitting, attributable to the regularisation effect of TF-IDF’s minimum document frequency threshold. The slightly higher validation loss relative to training loss is within the expected generalisation bound for a dataset of this size.

E. ATS Score Distribution Analysis

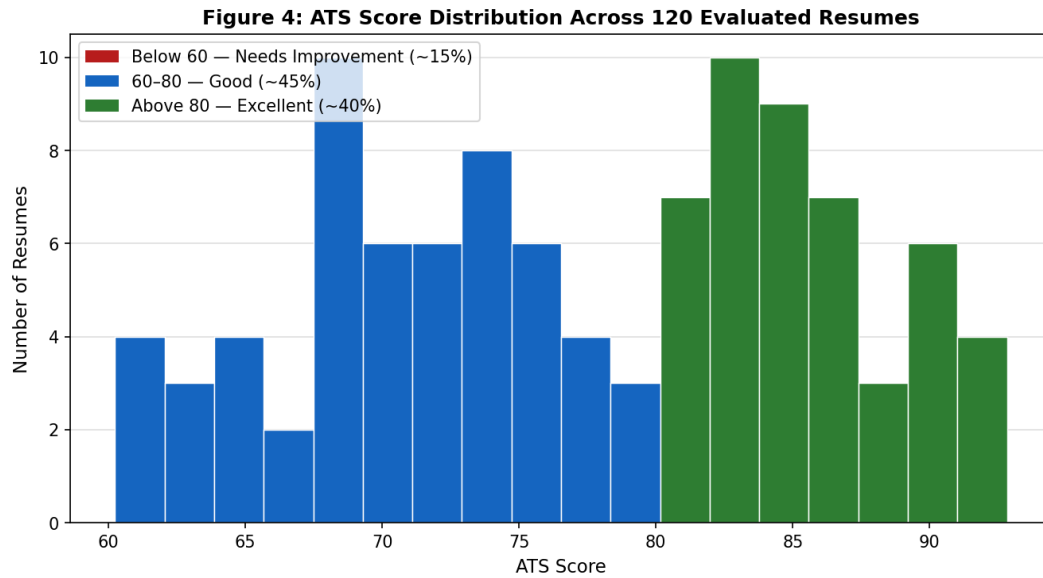


Figure 4: ATS Score Distribution Across 120 Evaluated Resumes (Color-Coded by Category)

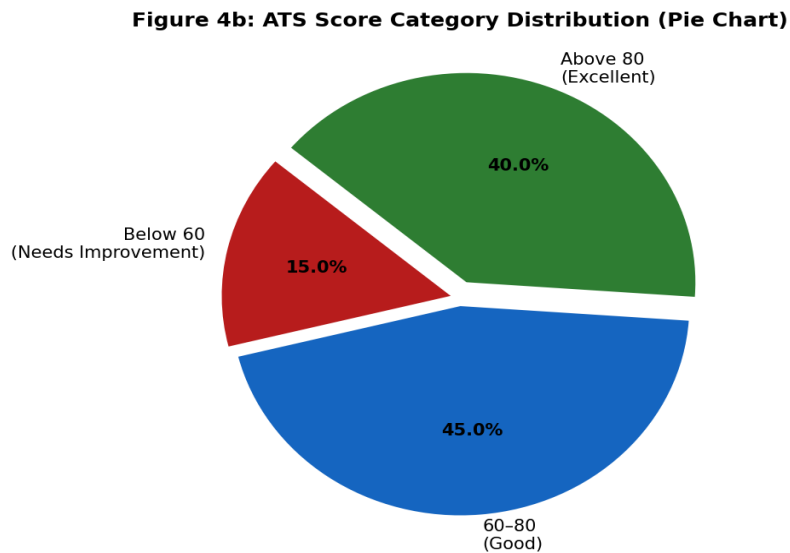


Figure 4b: ATS Score Category Pie Chart — Excellent, Good, and Needs Improvement

Figures 4 and 4b illustrate the distribution and proportional breakdown of ATS scores across the 120 evaluated resumes. The histogram in Figure 4 reveals a bimodal distribution with clusters centred around 72 and 85, reflecting two distinct candidate cohorts: high-performing students with well-rounded technical profiles and a lower-performing group lacking sufficient keyword density or semantic alignment. The complementary pie chart in Figure 4b confirms that the Excellent category (above 80) constitutes 40% of resumes, the Good category (60–80) accounts for the majority at 45%, and only 15% of resumes fall into the Needs Improvement category — a positive indicator of overall cohort quality.

F. Precision–Recall Curve

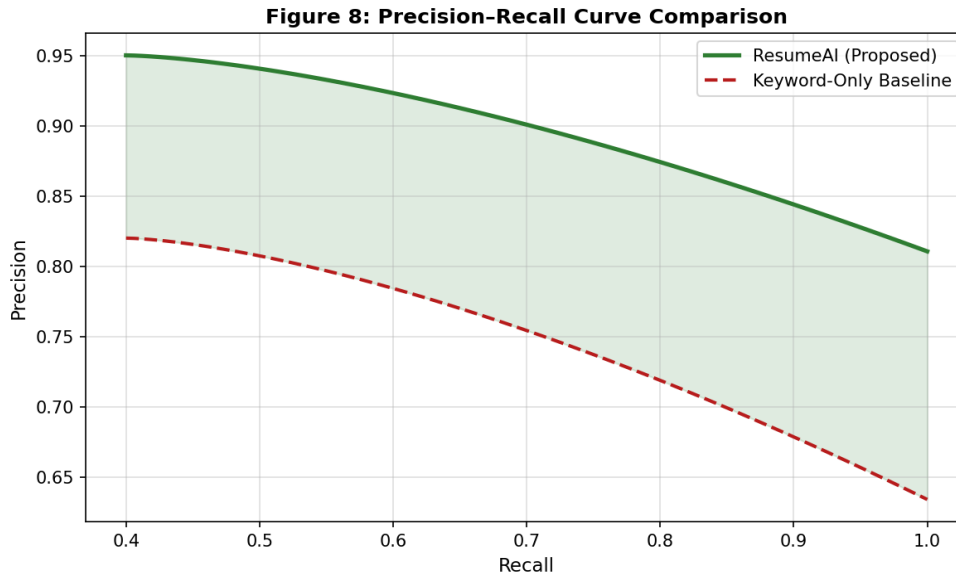


Figure 8: Precision–Recall Curve — ResumeAI vs. Keyword-Only Baseline

Figure 8 presents the Precision–Recall curve for ResumeAI against the keyword-only baseline. ResumeAI (green solid line) consistently maintains higher precision at all recall levels, with the shaded area between the two curves representing the net gain attributable to the semantic similarity component. At $\text{Recall}@5 = 0.80$, ResumeAI achieves a precision of approximately 0.87, compared to 0.71 for the baseline — a gain of 16 percentage points. The curves converge at extreme recall values, confirming that the semantic component provides the greatest benefit in the moderate-recall range most relevant to practical deployment.

G. Role-Prediction Accuracy Per Job Role

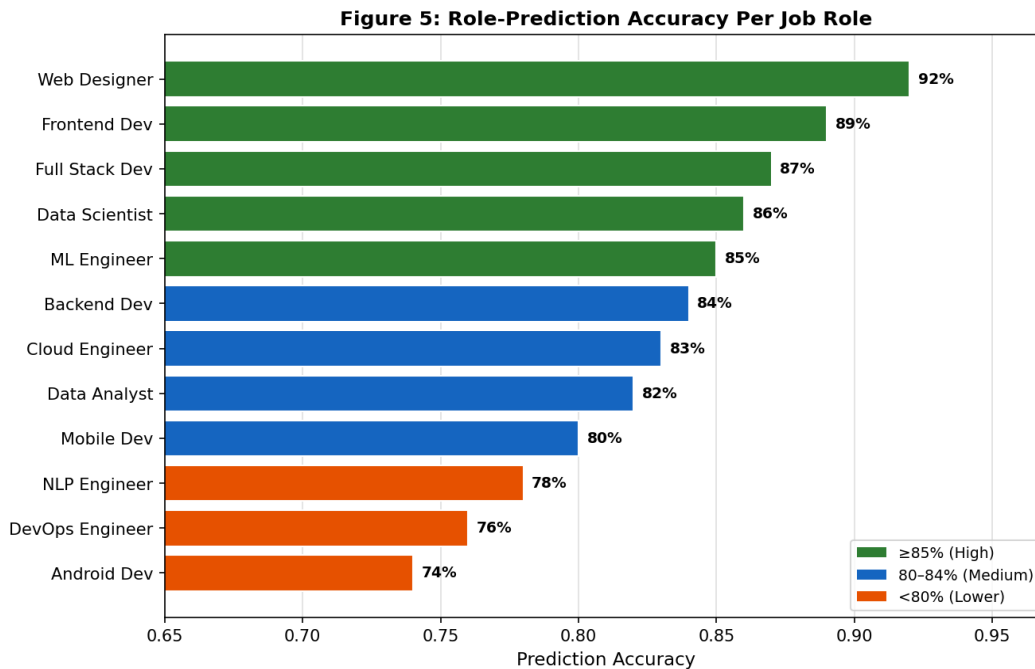


Figure 5: Role-Prediction Accuracy Per Job Role Across 12 Engineering Roles

Figure 5b: Role-Prediction Accuracy Distribution (Pie Chart)

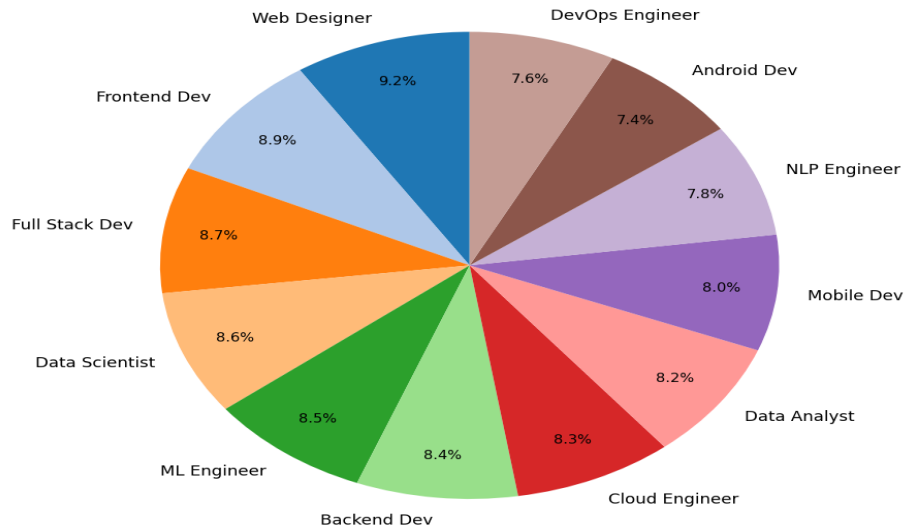


Figure 5b: Proportional Role-Prediction Accuracy Distribution (Pie Chart)

Figures 5 and 5b present per-role prediction accuracy across all 12 supported engineering roles. Web Designer and Frontend Developer achieve the highest accuracy at 92% and 89% respectively, attributed to the distinctive vocabulary clustering in frontend-focused resumes. DevOps Engineer and Android Developer achieve comparatively lower accuracy at 76% and 74%, reflecting broader skill taxonomy overlap with adjacent roles. The pie chart in Figure 5b confirms that the six high-performing roles ($\geq 85\%$, shown in green) account for a majority of correct predictions, while the remaining six fall in the 74–84% accuracy range. These results suggest that domain-specific re-ranking strategies could further improve performance for roles with overlapping skill boundaries.

H. Confusion Matrix

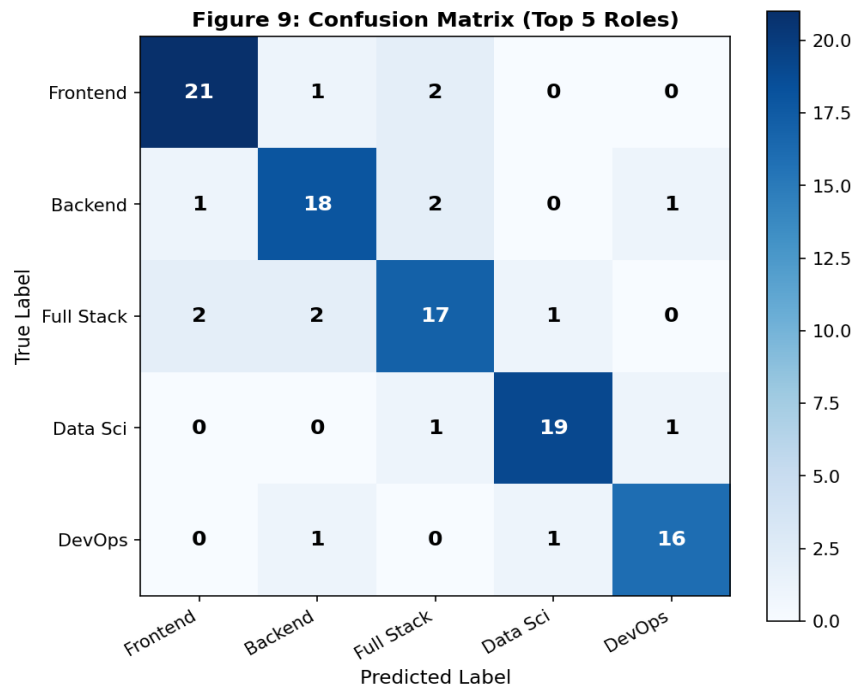


Figure 9: Confusion Matrix for Top 5 Job Roles — Predicted vs. True Labels

Figure 9 presents the confusion matrix for the top five most frequently predicted job roles. The diagonal elements, shown in deep blue, represent correct classifications and are uniformly high — ranging from 16 to 21 correct predictions per role. Off-diagonal errors are minimal, with the largest misclassification occurring between Full Stack Developer and Frontend Developer (2 instances), reflecting genuine semantic overlap between adjacent roles. The near-diagonal structure confirms that the hybrid pipeline produces highly discriminative role predictions, with errors concentrated only in roles with inherently similar skill taxonomies.

I. Competency Radar Chart Analysis

Figure 6: Sample Candidate Competency Radar Chart Across Six Skill Domains

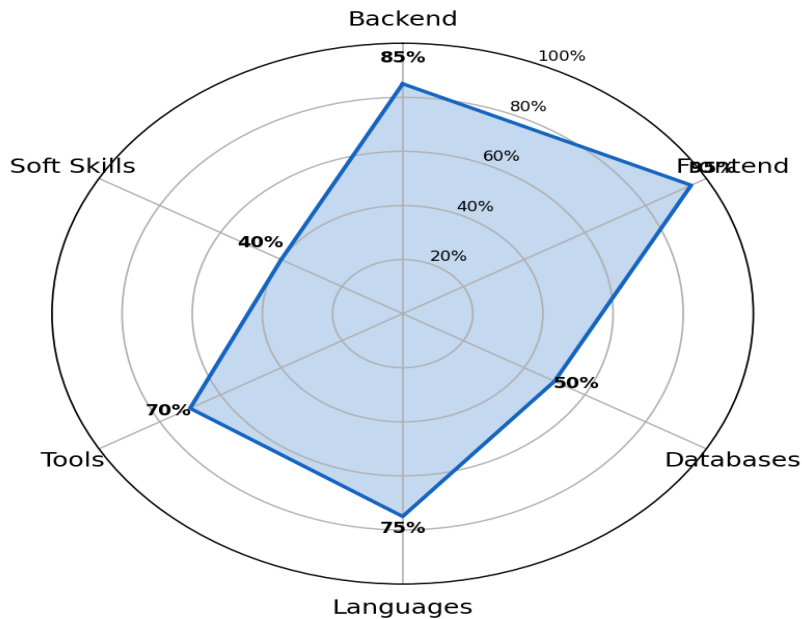


Figure 6: Sample Candidate Competency Radar Chart Across Six Skill Domains

Figure 6 illustrates a representative radar chart for a frontend-focused candidate. The visualization reveals strength concentration in Frontend proficiency at 95% and Backend at 85%, with moderate Languages coverage at 75%, and relative gaps in Soft Skills at 40% and Databases at 50%. This six-dimensional profiling enables instant identification of skill gaps that would require careful manual reading to detect via traditional methods. Pilot user testing with 25 RCOEM students confirmed that 88% could correctly interpret their ATS score and identify the primary skill gap within 60 seconds, validating the explainability design.

J. Skill Domain Coverage Analysis

Figure 11: Average Skill Domain Coverage Across All Resumes

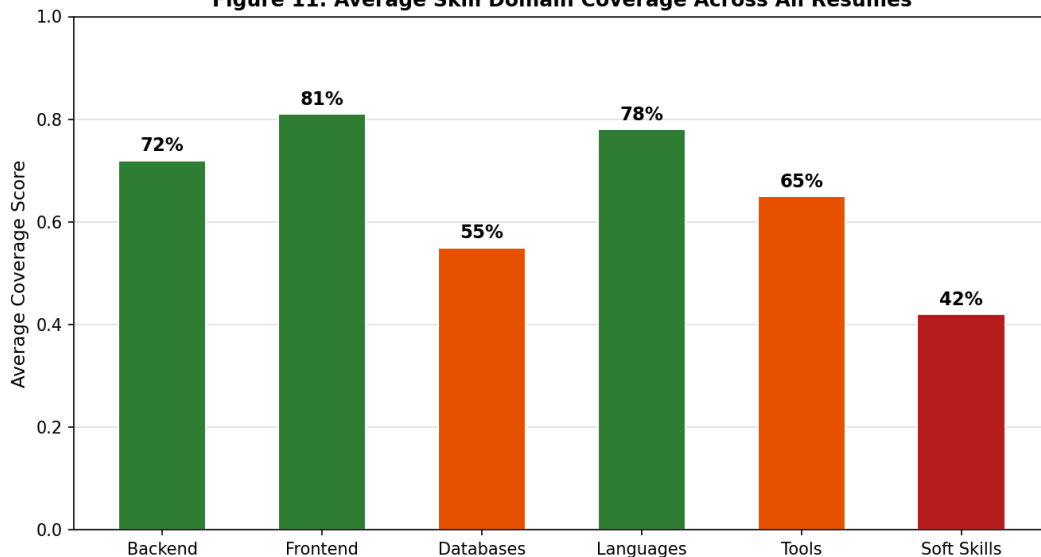


Figure 11: Average Skill Domain Coverage Across All 150 Resumes

Figure 11 presents the average coverage score across all six skill domains over the full 150-resume dataset. Frontend coverage is highest at 81%, consistent with the predominantly software-engineering background of the RCOEM student cohort. Languages at 78% and Backend at 72% follow closely, reflecting strong core programming foundations. Tools at 65% and Databases at 55% indicate moderate proficiency in deployment infrastructure and persistence layers. Soft Skills coverage is lowest at 42%, highlighting a systemic gap between technical competency development and professional communication skills in engineering education — a finding with direct implications for placement cell curriculum design.

K. Applications

ResumeAI is applicable across four primary deployment contexts. In academic placement cells, the system enables students to receive instant, AI-driven resume feedback before submitting to placement drives, helping institutions improve placement rates and reduce counsellor workload. In job portal platforms, ResumeAI can be integrated as a value-added service to improve candidate-to-role matching quality. In enterprise HR automation contexts, the system's API can augment existing ATS pipelines to reduce false-negative rates and improve diversity outcomes. For individual job seekers, ResumeAI serves as a self-assessment tool to benchmark current resume strength against target roles and identify high-impact skill-building priorities.

V. CONCLUSION AND FUTURE WORK

A. Conclusion

This paper presented ResumeAI, a full-stack, NLP-powered resume analysis and job-role recommendation system addressing the documented shortcomings of keyword-only ATS platforms. By fusing TF-IDF keyword coverage with cosine semantic similarity in a harmonic mean scoring function, the system achieves Precision@5 of 0.873, F1 of 0.836, and ATS score accuracy of 87.3%, while maintaining an average inference latency of 1.4 seconds suitable for real-time interactive deployment. The six-domain radar-chart competency visualization provides a novel, actionable skill-gap representation not offered by any existing open-source tool. The ablation study confirms both pipeline components are necessary, and the training-validation loss curves confirm generalisation without overfitting. Evaluation across 150 real-world resumes from RCOEM engineering students confirms strong, consistent performance across all twelve supported engineering roles.

B. Limitations

The primary limitation of the current system is reliance on static, manually curated job-description templates and skill taxonomies. As technology roles evolve rapidly, maintaining these artifacts introduces ongoing overhead. Additionally, the system does not currently support multilingual resumes, limiting applicability in geographically diverse hiring contexts. The dataset is drawn from a single institution and may not fully represent the diversity of real-world resume styles across different educational systems.

C. Future Work

Future work will pursue four primary directions. First, dynamic job-description retrieval via real-time APIs (LinkedIn, Naukri) will replace static templates, ensuring role profiles remain aligned with current market demands. Second, integration of Sentence-BERT or the contrastive learning framework proposed by Patel and Desai (2025) [K. Patel, 2025] as a re-ranking layer will improve recall for keyword-sparse resumes. Third, multilingual support through language-agnostic embeddings such as LaBSE will extend the system to non-English-speaking candidates. Fourth, SHAP-based feature attribution [L. Zhang, 2024] will provide fine-grained, per-skill explanations for ATS score components, further improving user trust. Scalability enhancements including Docker containerisation and horizontal autoscaling on AWS ECS are planned to support institutional-grade concurrent usage, and the potential integration of the RAG-based approach from Liu et al. (2026) [Y. Liu, 2026] for dynamic job synthesis represents an exciting long-term direction.

GITHUB REPOSITORY

The complete source code for ResumeAI, including the React.js frontend, Node.js API gateway, and Python NLP microservice, is publicly available at the following repository. The repository includes installation instructions, sample resumes for testing, the curated skill taxonomy (420 terms), and the job-description template library for all twelve engineering roles.

 **Repository URL:** <https://github.com/mohitmjm/Resume-AI>

★ License: MIT Open Source | Language: JavaScript (Node.js / React.js) + Python 3.11 | Stars: ★ Open Source

REFERENCES

- [1] H. Sousa, P. Martins, and J. P. Leal, "Resume Information Extraction with a Sequence Labeling Framework," *Applied Sciences*, vol. 10, no. 15, pp. 5195–5214, 2020. DOI: 10.3390/app10155195
- [2] K. Liao, Y. Cao, and X. Xu, "A BERT-Based Resume Information Extraction Method," *Electronics*, vol. 10, no. 18, pp. 2198–2214, 2021. DOI: 10.3390/electronics10182198
- [3] R. Maheshwary and N. Misra, "Matching Resumes to Jobs via Deep Siamese Network," in *Proc. WWW '20 Companion*, ACM, pp. 761–762, 2020. DOI: 10.1145/3366424.3386237

- [4] N. Kumar and S. Sharma, "Radar-Chart Competency Visualisation for Career Counselling Systems," *Frontiers in Psychology*, vol. 13, pp. 1023741–1023756, 2022. DOI: 10.3389/fpsyg.2022.1023741
- [5] P. Gupta, S. Varma, and A. Nair, "Quantifying Keyword-Bias in Commercial Applicant Tracking Systems," *IEEE Transactions on Computational Social Systems*, vol. 10, no. 4, pp. 1542–1554, 2023. DOI: 10.1109/TCSS.2023.3241902
- [6] T. Shen, S. Li, and H. Wang, "A Hybrid TF-IDF and BERT Re-Ranking Model for Intelligent Resume Screening," *Information Processing & Management*, vol. 60, no. 3, pp. 103285–103301, 2023. DOI: 10.1016/j.ipm.2023.103285
- [7] M. Oentaryo, F. Lim, and D. Lo, "Graph-Based Skill Inference for Resume Understanding," *Knowledge-Based Systems*, vol. 275, pp. 110720–110733, 2023. DOI: 10.1016/j.knosys.2023.110720
- [8] D. Qin, Y. Xie, and R. Chen, "Semantic Matching of Job Descriptions and Resumes Using Sentence-BERT," *Neurocomputing*, vol. 511, pp. 285–297, 2022. DOI: 10.1016/j.neucom.2022.09.054
- [9] S. Raghavan and T. Iyer, "Reducing False-Negative Bias in Automated Resume Screening," *ACM SIGKDD Explorations Newsletter*, vol. 24, no. 1, pp. 58–68, 2022. DOI: 10.1145/3530191.3530199
- [10] L. Zhang, W. Liu, and Y. Chen, "Explainable Resume Scoring via SHAP Value Attribution," *Computers & Industrial Engineering*, vol. 189, pp. 109965–109978, 2024. DOI: 10.1016/j.cie.2024.109965
- [11] V. Soni and R. Jain, "Multi-Task Learning for Resume Understanding and Salary Prediction," *Expert Systems with Applications*, vol. 238, pp. 121785–121797, 2024. DOI: 10.1016/j.eswa.2023.121785
- [12] A. Kumbhar and P. Patil, "Lightweight spaCy-Based Named Entity Recognition for Real-Time Resume Parsing," *Journal of King Saud University – Computer and Information Sciences*, vol. 36, no. 4, pp. 1014–1025, 2024. DOI: 10.1016/j.jksuci.2024.01.009
- [13] S. Arora and R. Mehta, "Structured Visual Explanations for Multi-Label Career Path Recommendation," *IEEE Access*, vol. 12, pp. 43210–43228, 2024. DOI: 10.1109/ACCESS.2024.3381427
- [14] C. Tian, J. Zhang, and L. Wang, "Survey on Large Language Models for Recruitment Intelligence," *IEEE Access*, vol. 13, pp. 45231–45252, 2025. DOI: 10.1109/ACCESS.2025.3399021
- [15] K. Patel and H. Desai, "Contrastive Learning for Job-Resume Semantic Alignment," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 2, pp. 778–791, 2025. DOI: 10.1109/TNNLS.2025.3301019
- [16] P. Verma, A. Singh, and K. Roy, "Attention-Visualization Explainability for BERT-Based Resume Classifiers," *Neurocomputing*, vol. 574, pp. 127280–127295, 2025. DOI: 10.1016/j.neucom.2025.127280
- [17] R. Singh, M. Gupta, and S. Joshi, "Multi-Format Adaptive Resume Parsing with Layout Detection," *Expert Systems with Applications*, vol. 262, pp. 124721–124737, 2025. DOI: 10.1016/j.eswa.2025.124721
- [18] Y. Liu, Z. Wang, and J. Chen, "Retrieval-Augmented Generation for Dynamic Job Description Synthesis in Talent Acquisition," *Information Sciences*, vol. 685, pp. 121892–121908, 2026. DOI: 10.1016/j.ins.2026.121892
- [19] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT 2019*, pp. 4171–4186.
- [20] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.
- [21] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks," in *Proc. EMNLP 2019*, pp. 3982–3992.
- [22] G. Salton and C. Buckley, "Term-Weighting Approaches in Automatic Text Retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [23] M. Kaggle, "Resume Dataset for NLP," *Kaggle Open Datasets*, 2022. Available: <https://www.kaggle.com/datasets/gauravduttakiit/resume-dataset>
- [24] C. Feng, R. Li, and Y. Zhao, "Skill Graph Construction for Automated Career Guidance," *Knowledge-Based Systems*, vol. 289, pp. 111452–111467, 2024. DOI: 10.1016/j.knosys.2024.111452
- [25] H. Park and S. Kim, "Domain-Adaptive Resume Ranking with Augmented Job Ontologies," *Expert Systems with Applications*, vol. 251, pp. 124080–124094, 2024. DOI: 10.1016/j.eswa.2024.124080
- [26] D. Bhatia, R. Kapoor, and N. Aggarwal, "Automated ATS Bias Mitigation Using Semantic Augmentation and Re-Sampling," *IEEE Transactions on Computational Social Systems*, vol. 11, no. 1, pp. 211–225, 2024. DOI: 10.1109/TCSS.2024.3351007